

```

In [1]: # =====
# Project:
# VGG-16 for Skin Disease Classification
#
# Description:
# An image classification pipeline using a pre-trained VGG-16 model to classify
# skin diseases. It preloads the entire dataset into RAM for faster training and
# includes early stopping.
#
# Author:
# Gemini (Modified from original ViT-GNN script by MD. Sazib Ahmed)
#
# Date:
# September 24, 2025
# =====

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
import math
import warnings
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

# Core ML/DL Libraries
import timm
from torchvision import transforms, datasets
from torch.utils.data import Dataset, DataLoader

# Evaluation & Utilities
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    cohen_kappa_score,
    matthews_corrcoef,
)
from sklearn.preprocessing import label_binarize

# Suppress non-critical warnings
warnings.filterwarnings("ignore", category=UserWarning, module="PIL")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

# =====
# I. SETUP AND CONFIGURATION

```

```

# =====
config = {
    # --- Hardware & Performance Tuning ---
    "DEVICE": torch.device("cuda" if torch.cuda.is_available() else "cpu"),
    "AMP_ENABLED": True,
    "NUM_WORKERS": 0, # MUST be 0 for in-memory dataset on Windows

    # --- File Paths ---
    "DATA_DIR": r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset",
    "RESULTS_DIR": "full_Dataset_VGG16",

    # --- Model & Training Hyperparameters ---
    "MODEL_NAME": 'vgg16',
    "IMAGE_SIZE": 224,
    "BATCH_SIZE": 128,
    "NUM_EPOCHS": 200,
    "LEARNING_RATE": 1e-4,
    "WEIGHT_DECAY": 1e-4,
    "WARMUP_EPOCHS": 10,
    "EARLY_STOPPING_PATIENCE": 20,

    # --- Reproducibility ---
    "RANDOM_SEED": 42
}

# --- Create results directory ---
os.makedirs(config["RESULTS_DIR"], exist_ok=True)
config["MODEL_SAVE_PATH_BEST"] = os.path.join(config["RESULTS_DIR"], "VGG16_best_mo

# --- Set Seed for Reproducible Experiments ---
def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True
    torch.backends.cudnn.deterministic = False

set_seed(config["RANDOM_SEED"])

# --- Initial Setup Verification ---
print("\n" + "="*80)
# <-- MODIFIED: Updated title to reflect VGG-16
print(" " * 20 + "VGG-16 Classification Pipeline Initialization")
print("="*80)
print(f"\n[INFO] PyTorch Version: {torch.__version__}")
print(f"[INFO] Data Loading Mode: In-Memory (Preloading to RAM)")
print(f"[INFO] Using Device: {config['DEVICE']}")
print(f"[INFO] Early Stopping Patience: {config['EARLY_STOPPING_PATIENCE']} epochs")
print(f"[INFO] Training for up to {config['NUM_EPOCHS']} epochs.")
print("="*80)

# =====

```

```

# II. DATA PREPARATION & HANDLING
# =====

# NEW: Custom Dataset for preloaded data
class InMemoryDataset(Dataset):
    def __init__(self, data_list, transform=None):
        """
        data_list: A list of tuples, where each tuple is (image_tensor, label_index)
        """
        self.data_list = data_list
        self.transform = transform

    def __len__(self):
        return len(self.data_list)

    def __getitem__(self, idx):
        image, label = self.data_list[idx]
        if self.transform:
            # Apply GPU-based augmentations if available
            image = self.transform(image)
        return image, label

# Define separate transforms for CPU pre-processing and potential GPU augmentation
# CPU transforms are applied once during pre-loading.
cpu_transforms = transforms.Compose([
    transforms.Resize((config["IMAGE_SIZE"], config["IMAGE_SIZE"])),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# GPU transforms can be applied on-the-fly during training for augmentation.
# This offloads augmentation work from the CPU.
gpu_train_transforms = torch.nn.Sequential(
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    # Note: ColorJitter is often faster on CPU but can be run on GPU
    # transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)
).to(config["DEVICE"])

print("\n" + "="*80)
print(" " * 28 + "Phase 1: Data Preparation")
print("="*80)

# MODIFIED: Preload all data into memory
try:
    # Use ImageFolder just to discover images and labels easily
    temp_datasets = {
        x: datasets.ImageFolder(os.path.join(config["DATA_DIR"], x))
        for x in ['train', 'val', 'test']
    }
    class_names = temp_datasets['train'].classes
    num_classes = len(class_names)

    print(f"[INFO] Discovered {num_classes} classes: {'', '.join(class_names)}")
    print("[INFO] Pre-loading all images into RAM. This may take a moment...")

```

```

preloaded_data = {'train': [], 'val': [], 'test': []}
for split in ['train', 'val', 'test']:
    for path, label in tqdm(temp_datasets[split].samples, desc=f"--> Loading {s
        with Image.open(path).convert("RGB") as img:
            image_tensor = cpu_transforms(img)
            preloaded_data[split].append((image_tensor, label))

# Create our custom in-memory datasets
image_datasets = {
    'train': InMemoryDataset(preloaded_data['train'], transform=cpu_train_trans
    'val': InMemoryDataset(preloaded_data['val'], # No augmentation for val/te
    'test': InMemoryDataset(preloaded_data['test'])
}

dataloaders = {
    x: DataLoader(
        image_datasets[x],
        batch_size=config["BATCH_SIZE"],
        shuffle=(x == 'train'),
        num_workers=config["NUM_WORKERS"],
        pin_memory=True # Effective when NUM_WORKERS > 0, but harmless
    )
    for x in ['train', 'val', 'test']
}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}

print(f"[SUCCESS] All image data has been pre-loaded into memory.")
print(f" - Dataset sizes: Train={dataset_sizes['train']}, Val={dataset_sizes['v

except FileNotFoundError as e:
    print(f"\n[ERROR] Dataset not found. Please check the path in config['DATA_DIR']
    print(f"Details: {e}")
    exit()

# =====
# III. MODEL, OPTIMIZER, AND SCHEDULER INITIALIZATION
# =====
# The timm library will now create a vgg16 model because config["MODEL_NAME"] was c
model = timm.create_model(config["MODEL_NAME"], pretrained=True, num_classes=num_cl
model.to(config["DEVICE"])

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=config["LEARNING_RATE"], weight_deca

lr_lambda = lambda epoch: (epoch + 1) / config["WARMUP_EPOCHS"] if epoch < config["
0.5 * (1 + math.cos(math.pi * (epoch - config["WARMUP_EPOCHS"])) / (config["NUM_
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

scaler = torch.amp.GradScaler(device=config["DEVICE"].type, enabled=config["AMP_ENA

# =====
# IV. TRAINING & VALIDATION LOOPS
# =====
def train_one_epoch():

```

```

model.train()
running_loss = 0.0
correct_preds = 0
progress_bar = tqdm(dataloaders['train'], desc="--> Training", leave=False)

for inputs, labels in progress_bar:
    inputs = inputs.to(config["DEVICE"], non_blocking=True)
    labels = labels.to(config["DEVICE"], non_blocking=True)

    # Apply GPU augmentations
    inputs = image_datasets['train'].transform(inputs)

    optimizer.zero_grad(set_to_none=True)

    with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
        outputs = model(inputs)
        loss = criterion(outputs, labels)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

    _, preds = torch.max(outputs, 1)
    running_loss += loss.item() * inputs.size(0)
    correct_preds += torch.sum(preds == labels.data)
    progress_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_loss = running_loss / dataset_sizes['train']
epoch_acc = correct_preds.double() / dataset_sizes['train']
return epoch_loss, epoch_acc.item()

def validate_one_epoch():
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['val'], desc="--> Validating", leave=False)

    with torch.no_grad():
        for inputs, labels in progress_bar:
            inputs = inputs.to(config["DEVICE"])
            labels = labels.to(config["DEVICE"])

            with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            correct_preds += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes['val']
    epoch_acc = correct_preds.double() / dataset_sizes['val']
    return epoch_loss, epoch_acc.item()

```

```

# =====
# V. EVALUATION & VISUALIZATION FUNCTIONS
# =====

def plot_training_history(history):
    save_path = os.path.join(config["RESULTS_DIR"], "training_history.png")
    epochs = range(1, len(history['train_loss']) + 1)
    plt.style.use('seaborn-v0_8-darkgrid')
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 7))
    ax1.plot(epochs, history['train_loss'], 'o-', label='Train Loss')
    ax1.plot(epochs, history['val_loss'], 'o-', label='Validation Loss')
    ax1.set_title('Model Loss Over Epochs')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Loss')
    ax1.legend()
    ax2.plot(epochs, [acc * 100 for acc in history['train_acc']], 'o-', label='Train Accuracy')
    ax2.plot(epochs, [acc * 100 for acc in history['val_acc']], 'o-', label='Validation Accuracy')
    ax2.set_title('Model Accuracy Over Epochs')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Accuracy (%)')
    ax2.legend()
    # <-- MODIFIED: Updated figure title to reflect VGG-16
    fig.suptitle('VGG-16 Training & Validation History')
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved training history plot to: {save_path}")

def plot_scalar_metrics(metrics):
    save_path = os.path.join(config["RESULTS_DIR"], "scalar_metrics.png")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
    sns.barplot(x='Value', y='Metric', data=metrics_df, ax=ax, palette='viridis', hatch=True)
    ax.set_title('Overall Scalar Performance Metrics')
    ax.set_xlabel('Value')
    ax.set_ylabel('')
    ax.set_xlim(0, 1)
    for p in ax.patches:
        width = p.get_width()
        ax.text(width + 0.01, p.get_y() + p.get_height() / 2, f'{width:.4f}', va='center')
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved scalar metrics plot to: {save_path}")

def perform_full_evaluation(model_path, loader, class_names):
    print("\n" + "="*80)
    print("OVERALL TEST PERFORMANCE")
    print("="*80)
    try:
        # The model is correctly loaded using the updated config["MODEL_NAME"]
        eval_model = timm.create_model(config["MODEL_NAME"], pretrained=False, num_classes=len(class_names))
    
```

```

eval_model.load_state_dict(torch.load(model_path, map_location=config["DEVICE"]))
eval_model.to(config["DEVICE"])
eval_model.eval()
except FileNotFoundError:
    print(f"\n[ERROR] Model file not found at {model_path}. Evaluation skipped.")
    return

all_labels, all_preds, all_probs = [], [], []
with torch.no_grad():
    for inputs, labels in tqdm(loader, desc="--> Evaluating on Test Set"):
        inputs = inputs.to(config["DEVICE"])
        outputs = eval_model(inputs)
        probs = F.softmax(outputs, dim=1)
        _, predicted = torch.max(outputs.data, 1)
        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())

all_labels, all_preds, all_probs = np.array(all_labels), np.array(all_preds), np.array(all_probs)
print(classification_report(all_labels, all_preds, target_names=class_names, digits=2))

labels_bin = label_binarize(all_labels, classes=range(len(class_names)))
macro_roc_auc = roc_auc_score(labels_bin, all_probs, average='macro', multi_class='ovo')

scalar_metrics = {
    "Accuracy": (all_labels == all_preds).mean(),
    "Macro ROC AUC": macro_roc_auc,
    "Cohen's Kappa": cohen_kappa_score(all_labels, all_preds),
    "Matthews CorrCoef": matthews_corrcoef(all_labels, all_preds),
}

print("\n--- Overall Scalar Metrics ---")
for name, value in scalar_metrics.items(): print(f"{name}: {value:.4f}")

cm_path = os.path.join(config["RESULTS_DIR"], "confusion_matrix.png")
plt.style.use('default')
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(all_labels, all_preds, normalize='true'), annot=True, cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Normalized Confusion Matrix')
plt.savefig(cm_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved normalized confusion matrix to: {cm_path}")

roc_path = os.path.join(config["RESULTS_DIR"], "roc_curves.png")
plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 9))
colors = matplotlib.colormaps.get('viridis', len(class_names))
for i, (class_name, color) in enumerate(zip(class_names, colors.colors)):
    fpr, tpr, _ = roc_curve(labels_bin[:, i], all_probs[:, i])
    plt.plot(fpr, tpr, color=color, lw=2.5, label=f'ROC for {class_name} (AUC = {roc_auc_score(labels_bin[:, i], all_probs[:, i]):.4f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Chance')
plt.xlim([-0.02, 1.0]); plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')

```

```

plt.title('ROC Curve - One-vs-Rest'); plt.legend(loc="lower right")
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"\nSaved ROC curves plot to: {roc_path}")

plot_scalar_metrics(scalar_metrics)
print("\nDone.")

# =====
# VI. MAIN EXECUTION BLOCK
# =====
if __name__ == '__main__':
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    epochs_no_improve = 0
    best_epoch = 0

    print("\n" + "="*80)
    print(" " * 29 + "Phase 2: Model Training")
    print("="*80)
    print(f"[INFO] Starting training for up to {config['NUM_EPOCHS']} epochs...")
    print(f"[INFO] Early stopping is active with patience={config['EARLY_STOPPING_P

for epoch in range(config["NUM_EPOCHS"]):
    train_loss, train_acc = train_one_epoch()
    val_loss, val_acc = validate_one_epoch()
    scheduler.step()

    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    current_lr = optimizer.param_groups[0]['lr']

    print(
        f"Epoch [{epoch+1:03d}/{config['NUM_EPOCHS']:03d}] | "
        f"LR: {current_lr:.2e} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
    )

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        best_epoch = epoch + 1
        torch.save(model.state_dict(), config["MODEL_SAVE_PATH_BEST"])
        print(f"    > Validation loss improved to {best_val_loss:.4f}. Saving b
    else:
        epochs_no_improve += 1
        print(f"    > Val loss did not improve for {epochs_no_improve} epoch(s)

    if epochs_no_improve >= config['EARLY_STOPPING_PATIENCE']:
        print(f"\n[INFO] Early stopping triggered after {epoch + 1} epochs.")
        print(f"    Best model was saved at epoch {best_epoch} with validati
        break

```

```
print(f"\n[SUCCESS] Model training completed.")
plot_training_history(history)

perform_full_evaluation(
    model_path=config["MODEL_SAVE_PATH_BEST"],
    loader=dataloaders['test'],
    class_names=class_names
)
```

VGG-16 Classification Pipeline Initialization

```
[INFO] PyTorch Version: 2.7.1+cu128
[INFO] Data Loading Mode: In-Memory (Preloading to RAM)
[INFO] Using Device: cuda
[INFO] Early Stopping Patience: 20 epochs
[INFO] Training for up to 200 epochs.
```

Phase 1: Data Preparation

```
[INFO] Discovered 5 classes: Herpes, Melanoma, Monkeypox, Sarampion, Varicela
[INFO] Pre-loading all images into RAM. This may take a moment...
```

```
--> Loading train data: 100%|██████████|  
2000/2000 [00:31<00:00, 63.11it/s]  
--> Loading val data: 100%|██████████|  
| 250/250 [00:01<00:00, 178.85it/s]  
--> Loading test data: 100%|██████████|  
| 500/500 [00:05<00:00, 83.57it/s]
```

```
[SUCCESS] All image data has been pre-loaded into memory.
- Dataset sizes: Train=2000, Val=250, Test=500
```

Phase 2: Model Training

```
[INFO] Starting training for up to 200 epochs...
[INFO] Early stopping is active with patience=20.
```

Epoch [001/200] | LR: 2.00e-05 | Train Loss: 1.3305 | Train Acc: 0.4975 | Val Loss: 1.2129 | Val Acc: 0.6160

```
> Validation loss improved to 1.2129. Saving best model.
```

Epoch [002/200] | LR: 3.00e-05 | Train Loss: 0.6496 | Train Acc: 0.7760 | Val Loss: 0.7590 | Val Acc: 0.7360

```
> Validation loss improved to 0.7590. Saving best model.
```

Epoch [003/200] | LR: 4.00e-05 | Train Loss: 0.3834 | Train Acc: 0.8680 | Val Loss: 0.5946 | Val Acc: 0.7840

```
> Validation loss improved to 0.5946. Saving best model.
```

Epoch [004/200] | LR: 5.00e-05 | Train Loss: 0.2255 | Train Acc: 0.9215 | Val Loss: 0.4511 | Val Acc: 0.8640

```
> Validation loss improved to 0.4511. Saving best model.
```

```
Epoch [005/200] | LR: 6.00e-05 | Train Loss: 0.1418 | Train Acc: 0.9535 | Val Loss: 0.4924 | Val Acc: 0.8600
> Val loss did not improve for 1 epoch(s).

Epoch [006/200] | LR: 7.00e-05 | Train Loss: 0.1122 | Train Acc: 0.9615 | Val Loss: 0.5164 | Val Acc: 0.8320
> Val loss did not improve for 2 epoch(s).

Epoch [007/200] | LR: 8.00e-05 | Train Loss: 0.0965 | Train Acc: 0.9690 | Val Loss: 0.6746 | Val Acc: 0.8040
> Val loss did not improve for 3 epoch(s).

Epoch [008/200] | LR: 9.00e-05 | Train Loss: 0.1314 | Train Acc: 0.9550 | Val Loss: 0.4645 | Val Acc: 0.8400
> Val loss did not improve for 4 epoch(s).

Epoch [009/200] | LR: 1.00e-04 | Train Loss: 0.0951 | Train Acc: 0.9700 | Val Loss: 0.9950 | Val Acc: 0.8120
> Val loss did not improve for 5 epoch(s).

Epoch [010/200] | LR: 1.00e-04 | Train Loss: 0.0977 | Train Acc: 0.9665 | Val Loss: 0.8033 | Val Acc: 0.8520
> Val loss did not improve for 6 epoch(s).

Epoch [011/200] | LR: 1.00e-04 | Train Loss: 0.0793 | Train Acc: 0.9730 | Val Loss: 0.6693 | Val Acc: 0.8240
> Val loss did not improve for 7 epoch(s).

Epoch [012/200] | LR: 1.00e-04 | Train Loss: 0.0656 | Train Acc: 0.9805 | Val Loss: 0.6066 | Val Acc: 0.8400
> Val loss did not improve for 8 epoch(s).

Epoch [013/200] | LR: 9.99e-05 | Train Loss: 0.0791 | Train Acc: 0.9750 | Val Loss: 0.4563 | Val Acc: 0.8160
> Val loss did not improve for 9 epoch(s).

Epoch [014/200] | LR: 9.99e-05 | Train Loss: 0.0439 | Train Acc: 0.9860 | Val Loss: 0.7547 | Val Acc: 0.8600
> Val loss did not improve for 10 epoch(s).

Epoch [015/200] | LR: 9.98e-05 | Train Loss: 0.0726 | Train Acc: 0.9725 | Val Loss: 0.6275 | Val Acc: 0.8160
> Val loss did not improve for 11 epoch(s).

Epoch [016/200] | LR: 9.98e-05 | Train Loss: 0.0369 | Train Acc: 0.9880 | Val Loss: 0.4824 | Val Acc: 0.8600
> Val loss did not improve for 12 epoch(s).

Epoch [017/200] | LR: 9.97e-05 | Train Loss: 0.0289 | Train Acc: 0.9895 | Val Loss: 0.7519 | Val Acc: 0.8480
> Val loss did not improve for 13 epoch(s).
```

```
Epoch [018/200] | LR: 9.96e-05 | Train Loss: 0.0285 | Train Acc: 0.9885 | Val Loss:
0.5526 | Val Acc: 0.8680
> Val loss did not improve for 14 epoch(s).
```

```
Epoch [019/200] | LR: 9.94e-05 | Train Loss: 0.0386 | Train Acc: 0.9850 | Val Loss: 0.6262 | Val Acc: 0.8720
> Val loss did not improve for 15 epoch(s).
```

```
Epoch [020/200] | LR: 9.93e-05 | Train Loss: 0.0270 | Train Acc: 0.9900 | Val Loss: 0.6369 | Val Acc: 0.8680
> Val loss did not improve for 16 epoch(s).
```

```
Epoch [021/200] | LR: 9.92e-05 | Train Loss: 0.0255 | Train Acc: 0.9905 | Val Loss: 0.6589 | Val Acc: 0.8640
> Val loss did not improve for 17 epoch(s).
```

```
Epoch [022/200] | LR: 9.90e-05 | Train Loss: 0.0566 | Train Acc: 0.9835 | Val Loss: 0.6423 | Val Acc: 0.8240
> Val loss did not improve for 18 epoch(s).
```

```
Epoch [023/200] | LR: 9.88e-05 | Train Loss: 0.0554 | Train Acc: 0.9825 | Val Loss: 0.6831 | Val Acc: 0.8600
> Val loss did not improve for 19 epoch(s).
```

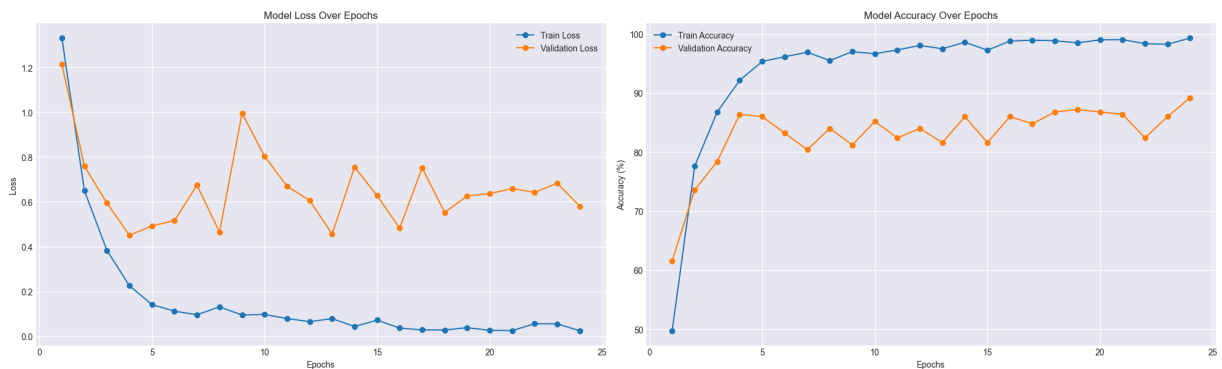
```
Epoch [024/200] | LR: 9.87e-05 | Train Loss: 0.0252 | Train Acc: 0.9930 | Val Loss: 0.5806 | Val Acc: 0.8920
> Val loss did not improve for 20 epoch(s).
```

```
[INFO] Early stopping triggered after 24 epochs.
```

Best model was saved at epoch 4 with validation loss: 0.4511

[SUCCESS] Model training completed.

VGG-16 Training & Validation History



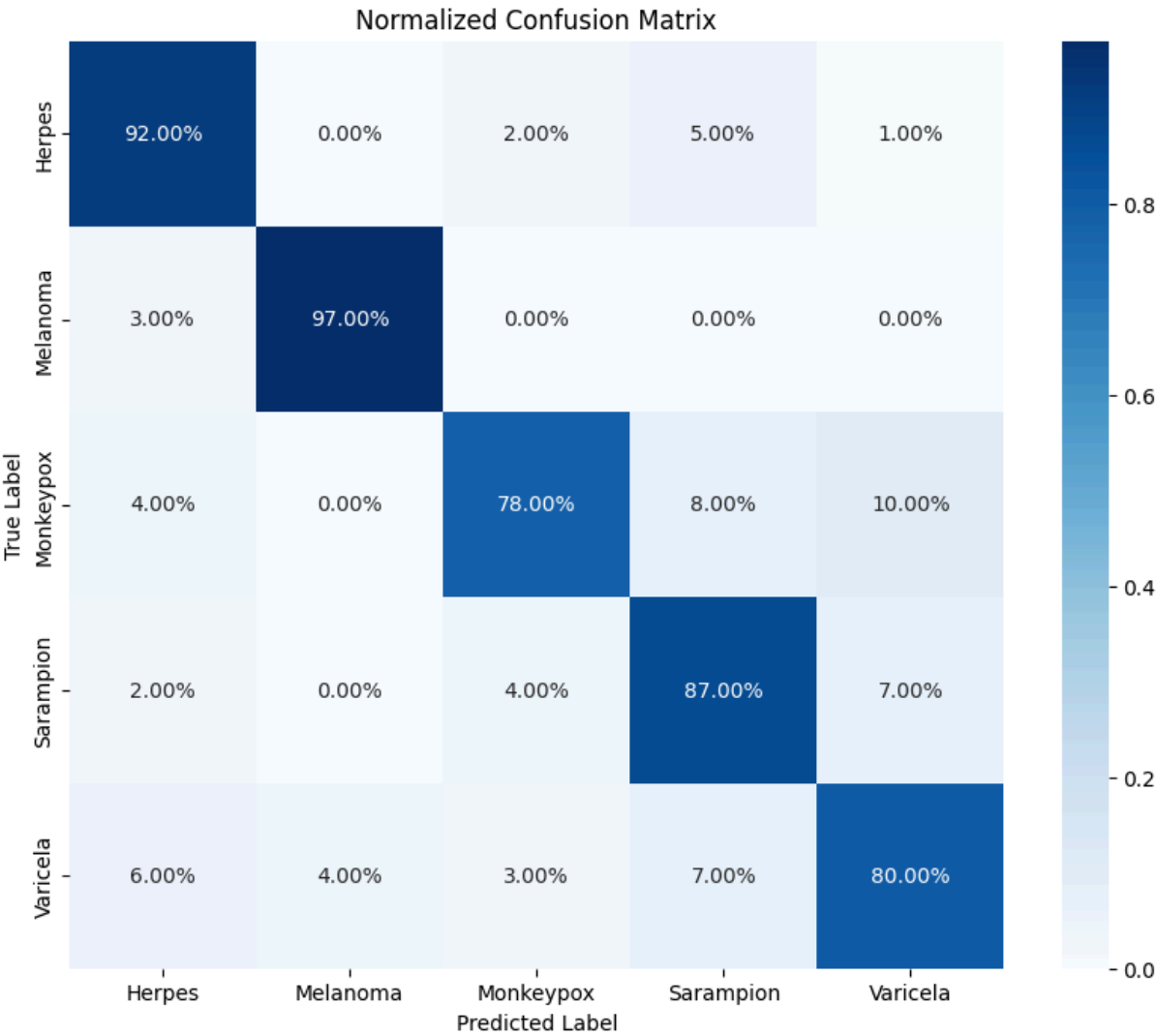
Saved training history plot to: full_results_VGG16_ImageOnly\training_history.png

OVERALL TEST PERFORMANCE

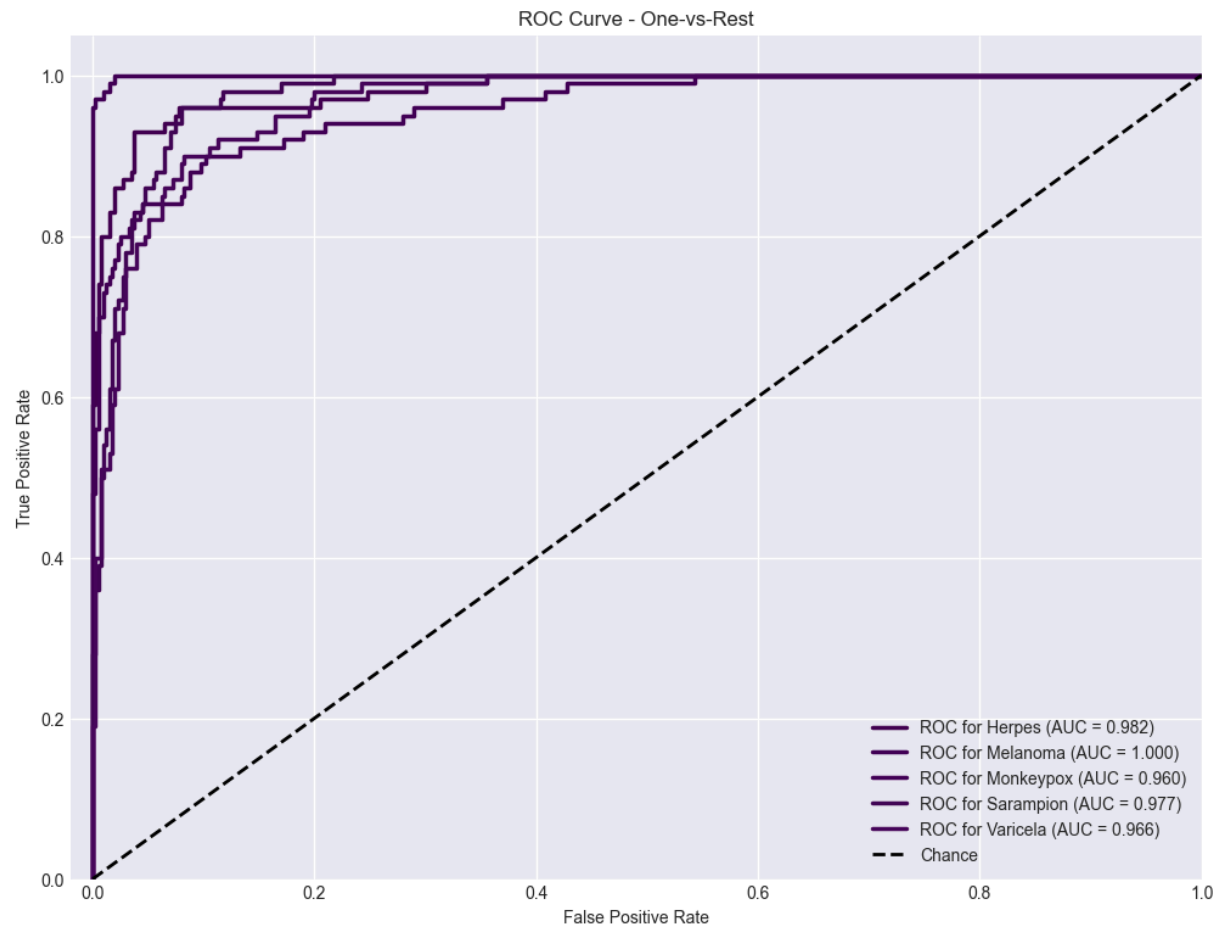
```
--> Evaluating on Test Set: 100%|██████████| 4/4 [00:11<00:00, 2.81s/it]
```

	precision	recall	f1-score	support
Herpes	0.86	0.92	0.89	100
Melanoma	0.96	0.97	0.97	100
Monkeypox	0.90	0.78	0.83	100
Sarampion	0.81	0.87	0.84	100
Varicela	0.82	0.80	0.81	100
accuracy			0.87	500
macro avg	0.87	0.87	0.87	500
weighted avg	0.87	0.87	0.87	500

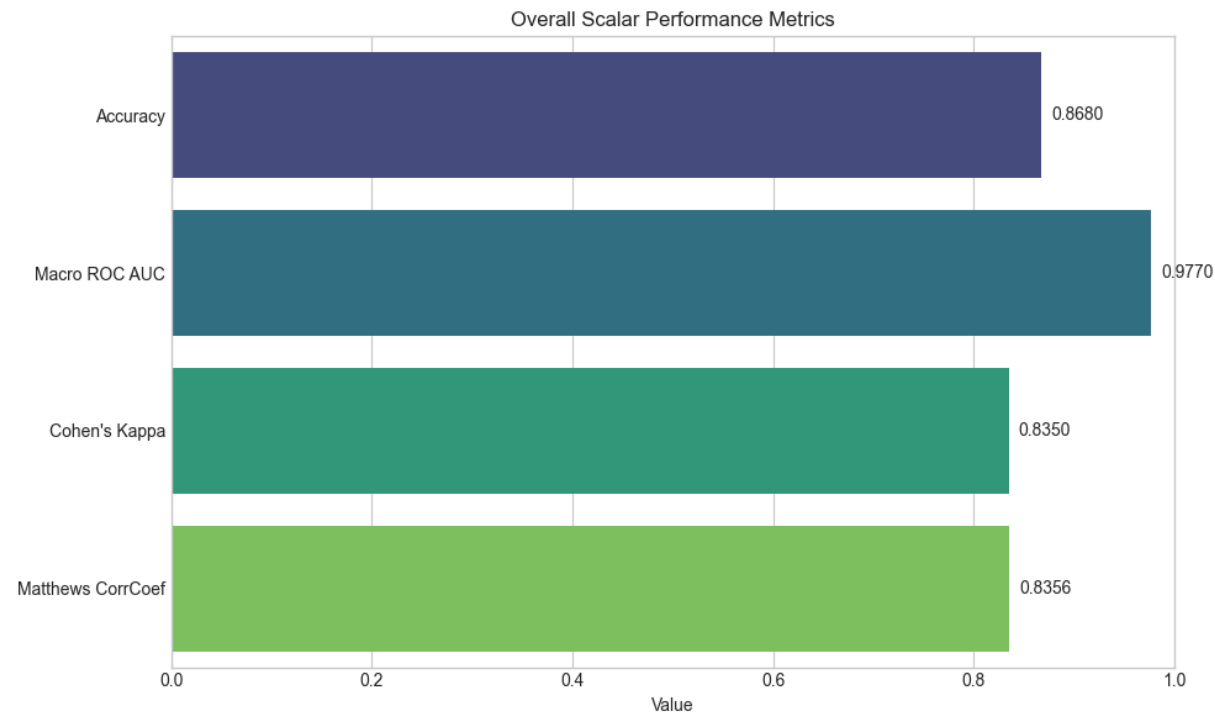
--- Overall Scalar Metrics ---
Accuracy: 0.8680
Macro ROC AUC: 0.9770
Cohen's Kappa: 0.8350
Matthews CorrCoef: 0.8356



Saved normalized confusion matrix to: full_results_VGG16_ImageOnly\confusion_matrix.png



Saved ROC curves plot to: full_results_VGG16_ImageOnly\roc_curves.png



Saved scalar metrics plot to: full_results_VGG16_ImageOnly\scalar_metrics.png

Done.

```
In [1]: # =====  
# Project:
```

```

# VGG16 for Skin Disease Classification
#
# Description:
# An image classification pipeline using a pre-trained VGG16 model to classify
# skin diseases. It preloads the entire dataset into RAM for faster training and
# includes early stopping and dropout.
#
# Author:
# Gemini (Modified from original ViT-GNN script by MD. Sazib Ahmed)
#
# Date:
# September 24, 2025
# =====

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
import math
import warnings
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

# Core ML/DL Libraries
import timm
from torchvision import transforms, datasets
from torch.utils.data import Dataset, DataLoader

# Evaluation & Utilities
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    cohen_kappa_score,
    matthews_corrcoef,
)
from sklearn.preprocessing import label_binarize

# Suppress non-critical warnings
warnings.filterwarnings("ignore", category=UserWarning, module="PIL")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

# =====
# I. SETUP AND CONFIGURATION
# =====

config = {
    # --- Hardware & Performance Tuning ---
    "DEVICE": torch.device("cuda" if torch.cuda.is_available() else "cpu"),

```

```

"AMP_ENABLED": True,
"NUM_WORKERS": 0, # MUST be 0 for in-memory dataset on Windows

# --- File Paths ---
"DATA_DIR": r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset",
"RESULTS_DIR": "full_results_VGG16_ImageOnly_dropout",

# --- Model & Training Hyperparameters ---
"MODEL_NAME": 'vgg16', # MODIFIED: Changed model to VGG16
"IMAGE_SIZE": 224,
"BATCH_SIZE": 128,
"NUM_EPOCHS": 200,
"LEARNING_RATE": 1e-4,
"WEIGHT_DECAY": 1e-4,
"WARMUP_EPOCHS": 10,
"EARLY_STOPPING_PATIENCE": 20,
"DROPOUT_RATE": 0.5, # ADDED: Dropout is often useful for VGG models

# --- Reproducibility ---
"RANDOM_SEED": 42
}

# --- Create results directory ---
os.makedirs(config["RESULTS_DIR"], exist_ok=True)
# MODIFIED: Updated model save path to reflect VGG16
config["MODEL_SAVE_PATH_BEST"] = os.path.join(config["RESULTS_DIR"], "VGG16_best_mo

# --- Set Seed for Reproducible Experiments ---
def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True
    torch.backends.cudnn.deterministic = False

set_seed(config["RANDOM_SEED"])

# --- Initial Setup Verification ---
print("\n" + "="*80)
# MODIFIED: Updated title to reflect VGG16
print(" " * 20 + "VGG16 Classification Pipeline Initialization")
print("="*80)
print(f"\n[INFO] PyTorch Version: {torch.__version__}")
print(f"[INFO] Data Loading Mode: In-Memory (Preloading to RAM)")
print(f"[INFO] Using Device: {config['DEVICE']}")
print(f"[INFO] Using Model: {config['MODEL_NAME']}")
print(f"[INFO] Dropout Rate: {config['DROPOUT_RATE']}")
print(f"[INFO] Early Stopping Patience: {config['EARLY_STOPPING_PATIENCE']} epochs")
print(f"[INFO] Training for up to {config['NUM_EPOCHS']} epochs.")
print("="*80)

# =====

```

```

# II. DATA PREPARATION & HANDLING
# =====

# Custom Dataset for preloaded data
class InMemoryDataset(Dataset):
    def __init__(self, data_list, transform=None):
        self.data_list = data_list
        self.transform = transform

    def __len__(self):
        return len(self.data_list)

    def __getitem__(self, idx):
        image, label = self.data_list[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

# Define separate transforms for CPU pre-processing and GPU augmentation
cpu_transforms = transforms.Compose([
    transforms.Resize((config["IMAGE_SIZE"], config["IMAGE_SIZE"])),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

gpu_train_transforms = torch.nn.Sequential(
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
).to(config["DEVICE"])

print("\n" + "="*80)
print(" " * 28 + "Phase 1: Data Preparation")
print("="*80)

# Preload all data into memory
try:
    temp_datasets = {
        x: datasets.ImageFolder(os.path.join(config["DATA_DIR"], x))
        for x in ['train', 'val', 'test']
    }
    class_names = temp_datasets['train'].classes
    num_classes = len(class_names)

    print(f"[INFO] Discovered {num_classes} classes: {' '.join(class_names)}")
    print("[INFO] Pre-loading all images into RAM. This may take a moment...")

    preloaded_data = {'train': [], 'val': [], 'test': []}
    for split in ['train', 'val', 'test']:
        for path, label in tqdm(temp_datasets[split].samples, desc=f"--> Loading {split}"):
            with Image.open(path).convert("RGB") as img:
                image_tensor = cpu_transforms(img)
                preloaded_data[split].append((image_tensor, label))

    image_datasets = {
        'train': InMemoryDataset(preloaded_data['train'], transform=gpu_train_trans

```

```

        'val': InMemoryDataset(preloaded_data['val']),
        'test': InMemoryDataset(preloaded_data['test'])
    }

    dataloaders = {
        x: DataLoader(
            image_datasets[x],
            batch_size=config["BATCH_SIZE"],
            shuffle=(x == 'train'),
            num_workers=config["NUM_WORKERS"],
            pin_memory=True
        )
        for x in ['train', 'val', 'test']
    }
    dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}

    print(f"[SUCCESS] All image data has been pre-loaded into memory.")
    print(f" - Dataset sizes: Train={dataset_sizes['train']}, Val={dataset_sizes['v

except FileNotFoundError as e:
    print(f"\n[ERROR] Dataset not found. Please check the path in config['DATA_DIR']")
    print(f"Details: {e}")
    exit()

# =====
# III. MODEL, OPTIMIZER, AND SCHEDULER INITIALIZATION
# =====
# MODIFIED: The only change needed is the model name. `timm` handles the rest.
model = timm.create_model(
    config["MODEL_NAME"],
    pretrained=True,
    num_classes=num_classes,
    drop_rate=config["DROPOUT_RATE"]
)
model.to(config["DEVICE"])

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=config["LEARNING_RATE"], weight_deca

lr_lambda = lambda epoch: (epoch + 1) / config["WARMUP_EPOCHS"] if epoch < config["
    0.5 * (1 + math.cos(math.pi * (epoch - config["WARMUP_EPOCHS"])) / (config["NUM_
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

scaler = torch.amp.GradScaler(device=config["DEVICE"].type, enabled=config["AMP_ENA

# =====
# IV. TRAINING & VALIDATION LOOPS
# =====
def train_one_epoch():
    model.train()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['train'], desc="--> Training", leave=False)

    for inputs, labels in progress_bar:

```

```

inputs = inputs.to(config["DEVICE"], non_blocking=True)
labels = labels.to(config["DEVICE"], non_blocking=True)

# Apply GPU augmentations
inputs = image_datasets['train'].transform(inputs)

optimizer.zero_grad(set_to_none=True)

with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
    outputs = model(inputs)
    loss = criterion(outputs, labels)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

_, preds = torch.max(outputs, 1)
running_loss += loss.item() * inputs.size(0)
correct_preds += torch.sum(preds == labels.data)
progress_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_loss = running_loss / dataset_sizes['train']
epoch_acc = correct_preds.double() / dataset_sizes['train']
return epoch_loss, epoch_acc.item()

def validate_one_epoch():
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['val'], desc="--> Validating", leave=False)

    with torch.no_grad():
        for inputs, labels in progress_bar:
            inputs = inputs.to(config["DEVICE"])
            labels = labels.to(config["DEVICE"])

            with torch.amp.autocast(device_type=config["DEVICE"].type, dtype=torch.float32):
                outputs = model(inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            correct_preds += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes['val']
    epoch_acc = correct_preds.double() / dataset_sizes['val']
    return epoch_loss, epoch_acc.item()

# =====
# V. EVALUATION & VISUALIZATION FUNCTIONS
# =====
def plot_training_history(history):
    save_path = os.path.join(config["RESULTS_DIR"], "training_history.png")
    epochs = range(1, len(history['train_loss']) + 1)

```

```

plt.style.use('seaborn-v0_8-darkgrid')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 7))
ax1.plot(epochs, history['train_loss'], 'o-', label='Train Loss')
ax1.plot(epochs, history['val_loss'], 'o-', label='Validation Loss')
ax1.set_title('Model Loss Over Epochs')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()
ax2.plot(epochs, [acc * 100 for acc in history['train_acc']], 'o-', label='Train Accuracy')
ax2.plot(epochs, [acc * 100 for acc in history['val_acc']], 'o-', label='Validation Accuracy')
ax2.set_title('Model Accuracy Over Epochs')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy (%)')
ax2.legend()
# MODIFIED: Updated title to reflect VGG16
fig.suptitle('VGG16 Training & Validation History', fontsize=20)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(save_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved training history plot to: {save_path}")

def plot_scalar_metrics(metrics):
    save_path = os.path.join(config["RESULTS_DIR"], "scalar_metrics.png")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
    sns.barplot(x='Value', y='Metric', data=metrics_df, ax=ax, palette='viridis', h
    ax.set_title('Overall Scalar Performance Metrics')
    ax.set_xlabel('Value')
    ax.set_ylabel('')
    ax.set_xlim(0, 1)
    for p in ax.patches:
        width = p.get_width()
        ax.text(width + 0.01, p.get_y() + p.get_height() / 2, f'{width:.4f}', va='c
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved scalar metrics plot to: {save_path}")

def perform_full_evaluation(model_path, loader, class_names):
    print("\n" + "="*80)
    print("OVERALL TEST PERFORMANCE")
    print("="*80)
    try:
        eval_model = timm.create_model(config["MODEL_NAME"], pretrained=False, num
        eval_model.load_state_dict(torch.load(model_path, map_location=config["DEVI
        eval_model.to(config["DEVICE"]))
        eval_model.eval()
    except FileNotFoundError:
        print(f"\n[ERROR] Model file not found at {model_path}. Evaluation skipped.
        return

```

```

all_labels, all_preds, all_probs = [], [], []
with torch.no_grad():
    for inputs, labels in tqdm(loader, desc="--> Evaluating on Test Set"):
        inputs = inputs.to(config["DEVICE"])
        outputs = eval_model(inputs)
        probs = F.softmax(outputs, dim=1)
        _, predicted = torch.max(outputs.data, 1)
        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())

all_labels, all_preds, all_probs = np.array(all_labels), np.array(all_preds), np.array(all_probs)
print(classification_report(all_labels, all_preds, target_names=class_names, digits=2))

labels_bin = label_binarize(all_labels, classes=range(len(class_names)))
macro_roc_auc = roc_auc_score(labels_bin, all_probs, average='macro', multi_class='ovo')

scalar_metrics = {
    "Accuracy": (all_labels == all_preds).mean(),
    "Macro ROC AUC": macro_roc_auc,
    "Cohen's Kappa": cohen_kappa_score(all_labels, all_preds),
    "Matthews CorrCoef": matthews_corrcoef(all_labels, all_preds),
}

print("\n--- Overall Scalar Metrics ---")
for name, value in scalar_metrics.items(): print(f"{name}: {value:.4f}")

cm_path = os.path.join(config["RESULTS_DIR"], "confusion_matrix.png")
plt.style.use('default')
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(all_labels, all_preds, normalize='true'), annot=True, cmap='magma')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Normalized Confusion Matrix')
plt.savefig(cm_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved normalized confusion matrix to: {cm_path}")

roc_path = os.path.join(config["RESULTS_DIR"], "roc_curves.png")
plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 9))
colors = matplotlib.colormaps.get('viridis', len(class_names))
for i, (class_name, color) in enumerate(zip(class_names, colors.colors)):
    fpr, tpr, _ = roc_curve(labels_bin[:, i], all_probs[:, i])
    plt.plot(fpr, tpr, color=color, lw=2.5, label=f'ROC for {class_name} (AUC = {roc_auc_score(labels_bin[:, i], all_probs[:, i]):.4f})')
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Chance')
plt.xlim([-0.02, 1.0]); plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve - One-vs-Rest'); plt.legend(loc="lower right")
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"\nSaved ROC curves plot to: {roc_path}")

plot_scalar_metrics(scalar_metrics)
print("\nDone.")

```

```

# =====
# VI. MAIN EXECUTION BLOCK
# =====
if __name__ == '__main__':
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    epochs_no_improve = 0
    best_epoch = 0

    print("\n" + "="*80)
    print(" " * 29 + "Phase 2: Model Training")
    print("="*80)
    print(f"[INFO] Starting training for up to {config['NUM_EPOCHS']} epochs...")
    print(f"[INFO] Early stopping is active with patience={config['EARLY_STOPPING_PATIENCE']}")

    for epoch in range(config["NUM_EPOCHS"]):
        train_loss, train_acc = train_one_epoch()
        val_loss, val_acc = validate_one_epoch()
        scheduler.step()

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

        current_lr = optimizer.param_groups[0]['lr']

        print(
            f"Epoch [{epoch+1:03d}/{config['NUM_EPOCHS']:03d}] | "
            f"LR: {current_lr:.2e} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
        )

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
            best_epoch = epoch + 1
            torch.save(model.state_dict(), config["MODEL_SAVE_PATH_BEST"])
            print(f"    > Validation loss improved to {best_val_loss:.4f}. Saving best model")
        else:
            epochs_no_improve += 1
            print(f"    > Val loss did not improve for {epochs_no_improve} epoch(s)")

        if epochs_no_improve >= config['EARLY_STOPPING_PATIENCE']:
            print(f"\n[INFO] Early stopping triggered after {epoch + 1} epochs.")
            print(f"    Best model was saved at epoch {best_epoch} with validation loss {best_val_loss:.4f}")
            break

    print(f"\n[SUCCESS] Model training completed.")
    plot_training_history(history)

    perform_full_evaluation(
        model_path=config["MODEL_SAVE_PATH_BEST"],
        loader=dataloaders['test'],

```

```
class_names=class_names
)
```

VGG16 Classification Pipeline Initialization

```
[INFO] PyTorch Version: 2.7.1+cu128
[INFO] Data Loading Mode: In-Memory (Preloading to RAM)
[INFO] Using Device: cuda
[INFO] Using Model: vgg16
[INFO] Dropout Rate: 0.5
[INFO] Early Stopping Patience: 20 epochs
[INFO] Training for up to 200 epochs.
```

Phase 1: Data Preparation

```
[INFO] Discovered 5 classes: Herpes, Melanoma, Monkeypox, Sarampion, Varicela
[INFO] Pre-loading all images into RAM. This may take a moment...
```

```
--> Loading train data: 100%|███████████  
2000/2000 [00:27<00:00, 71.50it/s]  
--> Loading val data: 100%|███████████  
| 250/250 [00:01<00:00, 192.95it/s]  
--> Loading test data: 100%|███████████  
| 500/500 [00:05<00:00, 87.64it/s]
```

```
[SUCCESS] All image data has been pre-loaded into memory.
- Dataset sizes: Train=2000, Val=250, Test=500
```

Phase 2: Model Training

```
[INFO] Starting training for up to 200 epochs...
[INFO] Early stopping is active with patience=20.
```

```
Epoch [001/200] | LR: 2.00e-05 | Train Loss: 1.5053 | Train Acc: 0.3610 | Val Loss: 1.4117 | Val Acc: 0.5040
  > Validation loss improved to 1.4117. Saving best model.
```

```
Epoch [002/200] | LR: 3.00e-05 | Train Loss: 1.0041 | Train Acc: 0.6225 | Val Loss:
0.8772 | Val Acc: 0.7080
> Validation loss improved to 0.8772. Saving best model.
```

```
Epoch [003/200] | LR: 4.00e-05 | Train Loss: 0.5955 | Train Acc: 0.7930 | Val Loss: 0.6805 | Val Acc: 0.7520
> Validation loss improved to 0.6805. Saving best model.
```

```
Epoch [004/200] | LR: 5.00e-05 | Train Loss: 0.3946 | Train Acc: 0.8615 | Val Loss: 0.5599 | Val Acc: 0.7880
  > Validation loss improved to 0.5599. Saving best model.
```

```
Epoch [005/200] | LR: 6.00e-05 | Train Loss: 0.2722 | Train Acc: 0.9015 | Val Loss:
0.4769 | Val Acc: 0.8240
  > Validation loss improved to 0.4769. Saving best model.
```

```
Epoch [006/200] | LR: 7.00e-05 | Train Loss: 0.2248 | Train Acc: 0.9190 | Val Loss: 0.4672 | Val Acc: 0.8200
  > Validation loss improved to 0.4672. Saving best model.

Epoch [007/200] | LR: 8.00e-05 | Train Loss: 0.2018 | Train Acc: 0.9295 | Val Loss: 0.4547 | Val Acc: 0.8440
  > Validation loss improved to 0.4547. Saving best model.

Epoch [008/200] | LR: 9.00e-05 | Train Loss: 0.1648 | Train Acc: 0.9420 | Val Loss: 0.3669 | Val Acc: 0.8680
  > Validation loss improved to 0.3669. Saving best model.

Epoch [009/200] | LR: 1.00e-04 | Train Loss: 0.1317 | Train Acc: 0.9600 | Val Loss: 0.4285 | Val Acc: 0.8640
  > Val loss did not improve for 1 epoch(s).

Epoch [010/200] | LR: 1.00e-04 | Train Loss: 0.1006 | Train Acc: 0.9640 | Val Loss: 0.4529 | Val Acc: 0.8640
  > Val loss did not improve for 2 epoch(s).

Epoch [011/200] | LR: 1.00e-04 | Train Loss: 0.1028 | Train Acc: 0.9640 | Val Loss: 0.3818 | Val Acc: 0.8960
  > Val loss did not improve for 3 epoch(s).

Epoch [012/200] | LR: 1.00e-04 | Train Loss: 0.1104 | Train Acc: 0.9650 | Val Loss: 0.3455 | Val Acc: 0.9120
  > Validation loss improved to 0.3455. Saving best model.

Epoch [013/200] | LR: 9.99e-05 | Train Loss: 0.1226 | Train Acc: 0.9595 | Val Loss: 0.4641 | Val Acc: 0.8640
  > Val loss did not improve for 1 epoch(s).

Epoch [014/200] | LR: 9.99e-05 | Train Loss: 0.0955 | Train Acc: 0.9670 | Val Loss: 0.4583 | Val Acc: 0.8600
  > Val loss did not improve for 2 epoch(s).

Epoch [015/200] | LR: 9.98e-05 | Train Loss: 0.0441 | Train Acc: 0.9840 | Val Loss: 0.6181 | Val Acc: 0.8720
  > Val loss did not improve for 3 epoch(s).

Epoch [016/200] | LR: 9.98e-05 | Train Loss: 0.0437 | Train Acc: 0.9830 | Val Loss: 0.6258 | Val Acc: 0.8760
  > Val loss did not improve for 4 epoch(s).

Epoch [017/200] | LR: 9.97e-05 | Train Loss: 0.0450 | Train Acc: 0.9835 | Val Loss: 0.5856 | Val Acc: 0.8800
  > Val loss did not improve for 5 epoch(s).

Epoch [018/200] | LR: 9.96e-05 | Train Loss: 0.0667 | Train Acc: 0.9775 | Val Loss: 0.5193 | Val Acc: 0.8720
  > Val loss did not improve for 6 epoch(s).
```

```
Epoch [019/200] | LR: 9.94e-05 | Train Loss: 0.0484 | Train Acc: 0.9875 | Val Loss: 0.8237 | Val Acc: 0.8000
> Val loss did not improve for 7 epoch(s).

Epoch [020/200] | LR: 9.93e-05 | Train Loss: 0.0520 | Train Acc: 0.9815 | Val Loss: 0.5182 | Val Acc: 0.8360
> Val loss did not improve for 8 epoch(s).

Epoch [021/200] | LR: 9.92e-05 | Train Loss: 0.0397 | Train Acc: 0.9875 | Val Loss: 0.5413 | Val Acc: 0.8480
> Val loss did not improve for 9 epoch(s).

Epoch [022/200] | LR: 9.90e-05 | Train Loss: 0.0358 | Train Acc: 0.9860 | Val Loss: 0.5564 | Val Acc: 0.8720
> Val loss did not improve for 10 epoch(s).

Epoch [023/200] | LR: 9.88e-05 | Train Loss: 0.0370 | Train Acc: 0.9835 | Val Loss: 0.5559 | Val Acc: 0.8880
> Val loss did not improve for 11 epoch(s).

Epoch [024/200] | LR: 9.87e-05 | Train Loss: 0.0496 | Train Acc: 0.9860 | Val Loss: 0.4677 | Val Acc: 0.8720
> Val loss did not improve for 12 epoch(s).

Epoch [025/200] | LR: 9.85e-05 | Train Loss: 0.0535 | Train Acc: 0.9830 | Val Loss: 0.7221 | Val Acc: 0.8440
> Val loss did not improve for 13 epoch(s).

Epoch [026/200] | LR: 9.83e-05 | Train Loss: 0.0430 | Train Acc: 0.9840 | Val Loss: 0.6120 | Val Acc: 0.8960
> Val loss did not improve for 14 epoch(s).

Epoch [027/200] | LR: 9.80e-05 | Train Loss: 0.0423 | Train Acc: 0.9850 | Val Loss: 0.7397 | Val Acc: 0.8480
> Val loss did not improve for 15 epoch(s).

Epoch [028/200] | LR: 9.78e-05 | Train Loss: 0.0257 | Train Acc: 0.9910 | Val Loss: 0.6333 | Val Acc: 0.8720
> Val loss did not improve for 16 epoch(s).

Epoch [029/200] | LR: 9.76e-05 | Train Loss: 0.0369 | Train Acc: 0.9885 | Val Loss: 0.5255 | Val Acc: 0.8800
> Val loss did not improve for 17 epoch(s).

Epoch [030/200] | LR: 9.73e-05 | Train Loss: 0.0204 | Train Acc: 0.9935 | Val Loss: 0.5117 | Val Acc: 0.8800
> Val loss did not improve for 18 epoch(s).

Epoch [031/200] | LR: 9.70e-05 | Train Loss: 0.0187 | Train Acc: 0.9930 | Val Loss: 0.5487 | Val Acc: 0.8680
> Val loss did not improve for 19 epoch(s).
```

Epoch [032/200] | LR: 9.67e-05 | Train Loss: 0.0285 | Train Acc: 0.9910 | Val Loss: 0.6177 | Val Acc: 0.8640

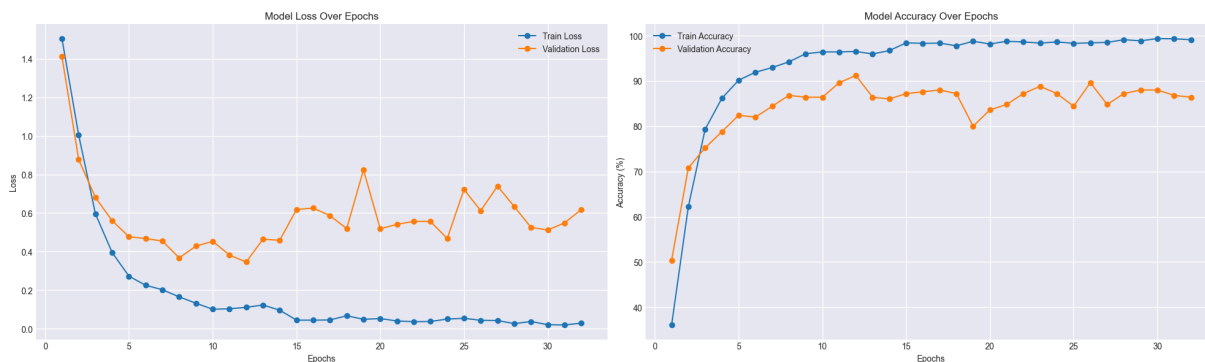
```
> Val loss did not improve for 20 epoch(s).
```

```
[INFO] Early stopping triggered after 32 epochs.
```

```
Best model was saved at epoch 12 with validation loss: 0.3455
```

[SUCCESS] Model training completed.

VGG16 Training & Validation History



Saved training history plot to: full_results_VGG16_ImageOnly_dropout\training_history.png

OVERALL TEST PERFORMANCE

```
--> Evaluating on Test Set: 100%|██████████| 4/4 [00:11<00:00, 2.77s/it]
```

	precision	recall	f1-score	support
Herpes	0.98	0.84	0.90	100
Melanoma	1.00	0.99	0.99	100
Monkeypox	0.80	0.84	0.82	100
Sarampion	0.79	0.92	0.85	100
Varicela	0.84	0.78	0.81	100
accuracy			0.87	500
macro avg	0.88	0.87	0.87	500
weighted avg	0.88	0.87	0.87	500

```

--- Overall Scalar Metrics ---

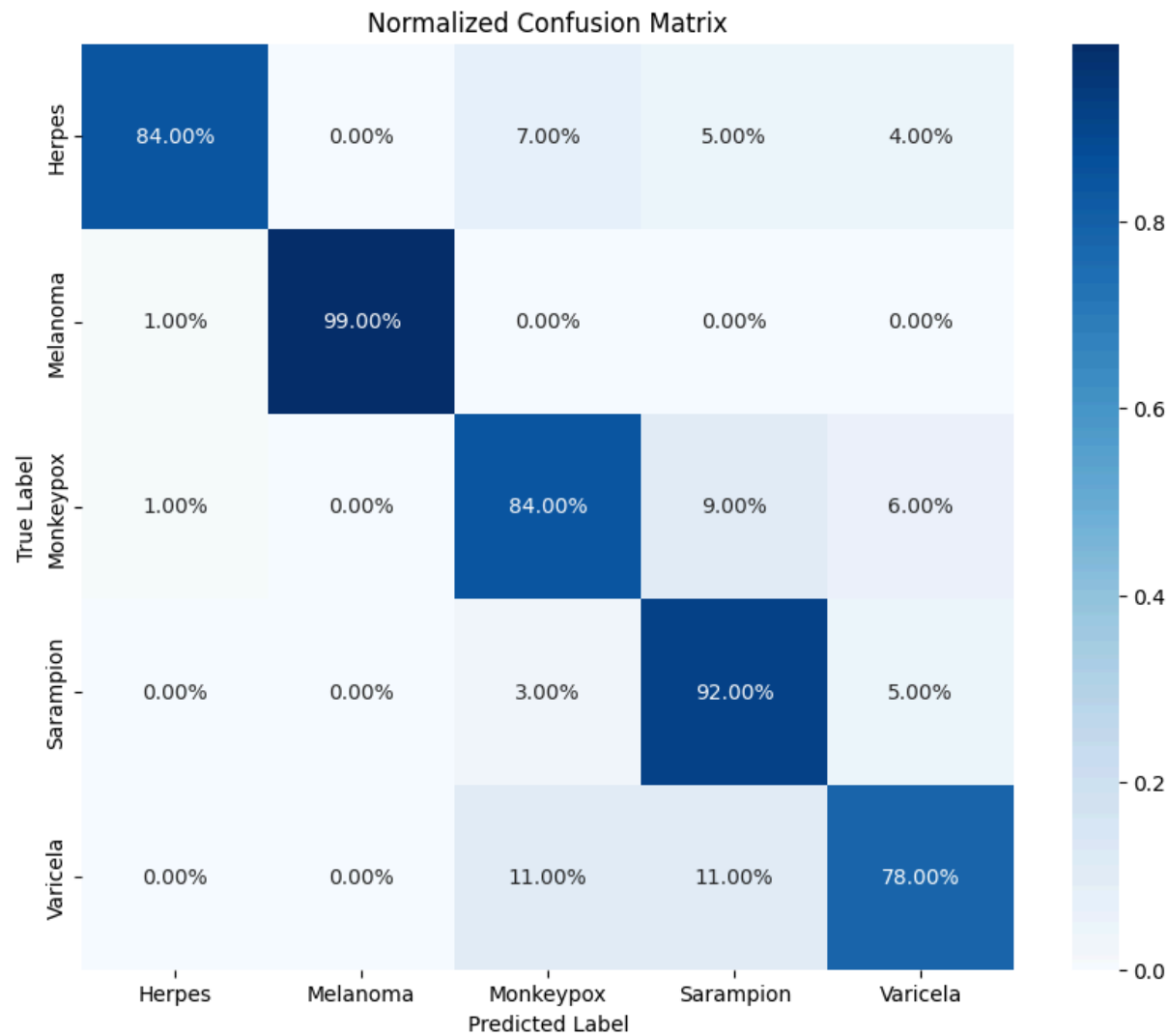
```

Accuracy: 0.8740

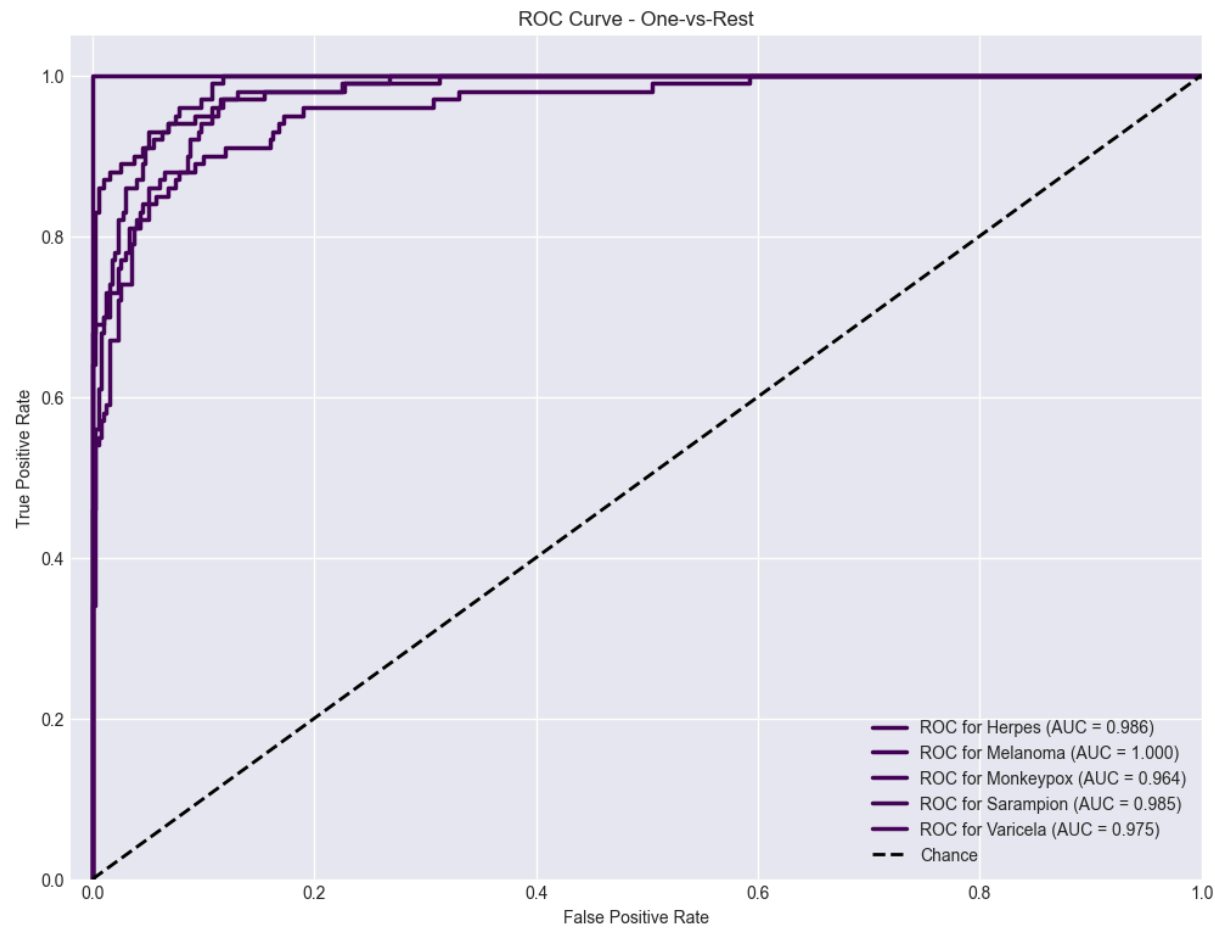
Macro ROC AUC: 0.9822

Cohen's Kappa: 0.8425

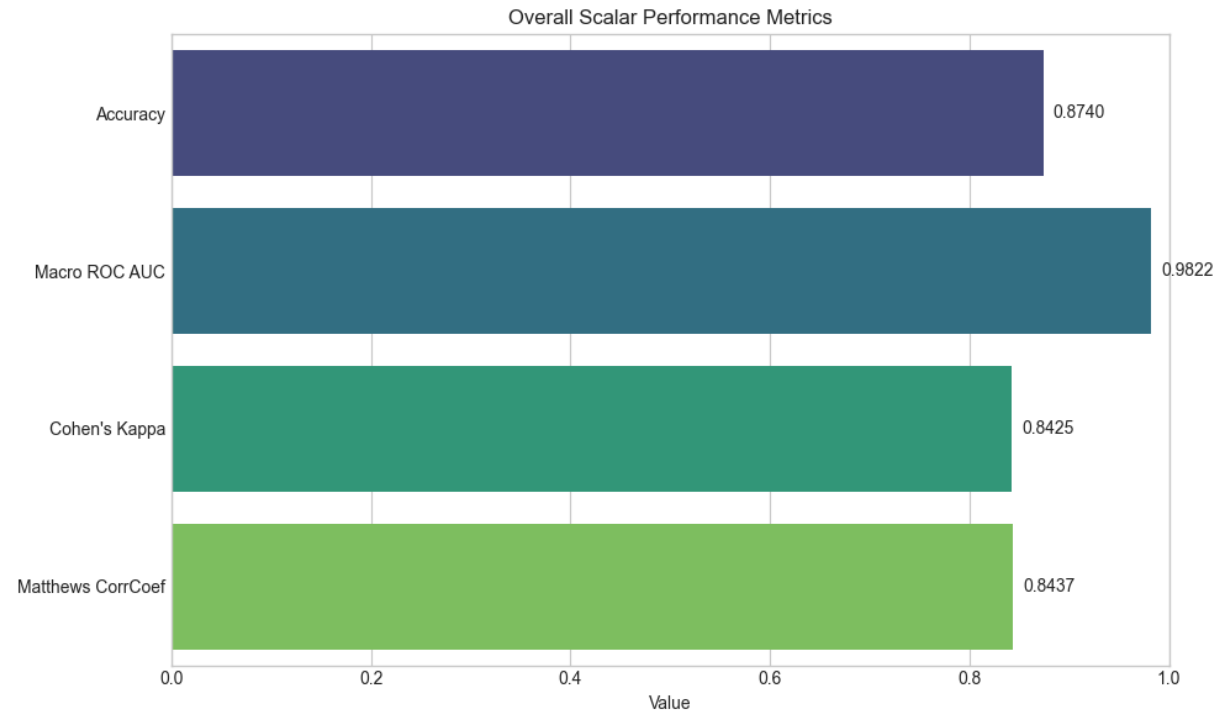
Matthews CorrCoef: 0.8437



Saved normalized confusion matrix to: full_results_VGG16_ImageOnly_dropout\confusion_matrix.png



Saved ROC curves plot to: full_results_VGG16_ImageOnly_dropout\roc_curves.png



Saved scalar metrics plot to: full_results_VGG16_ImageOnly_dropout\scalar_metrics.png

Done.

In []: